AWS Infrastructure Automation

Using Terraform & GitHub Actions

Project Submission: Terraform Configuration and AWS Console Evidence

GitHub Repository: https://github.com/Bhagyashree2993/aws-terraform-gha

AWS Region: Asia Pacific (Mumbai) – ap-south-1

Contents

1. Objective

2. Overall Workflow

3. AWS Resources Provisioned

4. Terraform Configuration

5. AWS Console Evidence (Screenshots 1–11)

6. GitHub Actions Pipeline Evidence (Screenshots 12–15)

7. Terraform Outputs

8. Security and Best Practices

9. Conclusion

Appendix: Terraform Configuration (Source Code)

1. Objective

The objective of this project is to provision AWS infrastructure using Terraform (Infrastructure as Code) and to automate its deployment through a GitHub Actions CI/CD pipeline. Every change to the infrastructure code is automatically formatted, validated and planned, and is applied to AWS only after a manual approval. After deployment, the pipeline publishes the Terraform outputs and automatically validates that the created resources are working.

2. Overall Workflow

GitHub → GitHub Actions → Format → Init → Validate → Plan → Plan Artifact → Approval → Apply → AWS Infrastructure → Outputs → Validation

#	Stage	What happens
1	GitHub	Code is pushed to the main branch (or a pull request is opened) and changes files in infra/.
2	GitHub Actions	The workflow starts and connects to AWS using credentials stored as GitHub secrets.
3	Format	terraform fmt -check verifies that all code is correctly formatted.
4	Init	terraform init downloads the AWS provider and connects to the S3 remote state backend.
5	Validate	terraform validate checks the syntax and configuration for errors.
6	Plan	terraform plan -out=tfplan calculates exactly which resources will be created or changed.
7	Plan Artifact	The saved plan file is uploaded as a downloadable artifact of the run.
8	Approval	The Apply job uses the "production" environment, which waits for a required reviewer to approve.
9	Apply	terraform apply tfplan applies exactly the plan that was reviewed and approved.
10	AWS Infrastructure	VPC, subnets, internet gateway, route table, security group, EC2, IAM role and S3 bucket are created.
11	Outputs	terraform output publishes resource IDs and the website URL in the run summary.
12	Validation	The pipeline checks the VPC state, EC2 status checks, S3 bucket access and that the website returns HTTP 200.

3. AWS Resources Provisioned

All resources below are created by Terraform and tagged with ManagedBy = Terraform, Project = tf-gha-demo and Environment = dev.

AWS Resource	Name in AWS	Terraform file
VPC (10.0.0.0/16)	tf-gha-demo-dev-vpc	infra/network.tf
2 Public Subnets	tf-gha-demo-dev-public-1, -public-2	infra/network.tf
Internet Gateway	tf-gha-demo-dev-igw	infra/network.tf

AWS Resource	Name in AWS	Terraform file
Route Table (0.0.0.0/0 → IGW)	tf-gha-demo-dev-public-rt	infra/network.tf
Security Group (HTTP 80)	tf-gha-demo-dev-web-sg	infra/network.tf
EC2 Instance (t3.micro, nginx)	tf-gha-demo-dev-web	infra/compute.tf
IAM Role + Instance Profile	tf-gha-demo-dev-web-role	infra/compute.tf
S3 Application Bucket	tf-gha-demo-dev-app-<account-id>	infra/storage.tf
S3 Remote State Bucket	tfstate-778436624052-ap-south-1	bootstrap/main.tf

4. Terraform Configuration

The complete configuration is available in the GitHub repository: https://github.com/Bhagyashree2993/aws-terraform-gha. The full source code of every file is included in the Appendix.

File / Folder	Purpose
infra/versions.tf	Terraform and AWS provider versions, S3 remote backend, default tags
infra/variables.tf	Input variables (region, CIDR ranges, instance type, etc.)
infra/terraform.tfvars	Values for the input variables
infra/network.tf	VPC, subnets, internet gateway, route table, security group
infra/compute.tf	EC2 web server, IAM role and instance profile, latest AMI lookup
infra/user_data.sh.tftpl	Startup script that installs nginx and creates the web page
infra/storage.tf	Private, encrypted, versioned S3 application bucket
infra/outputs.tf	Outputs: VPC ID, subnet IDs, instance ID, public IP, website URL, bucket name
bootstrap/main.tf	One-time setup: S3 bucket for Terraform remote state
.github/workflows/terraform.yml	CI/CD pipeline implementing all workflow stages
.github/workflows/terraform-destroy.yml	Manual workflow to destroy all resources (also needs approval)

5. AWS Console Evidence

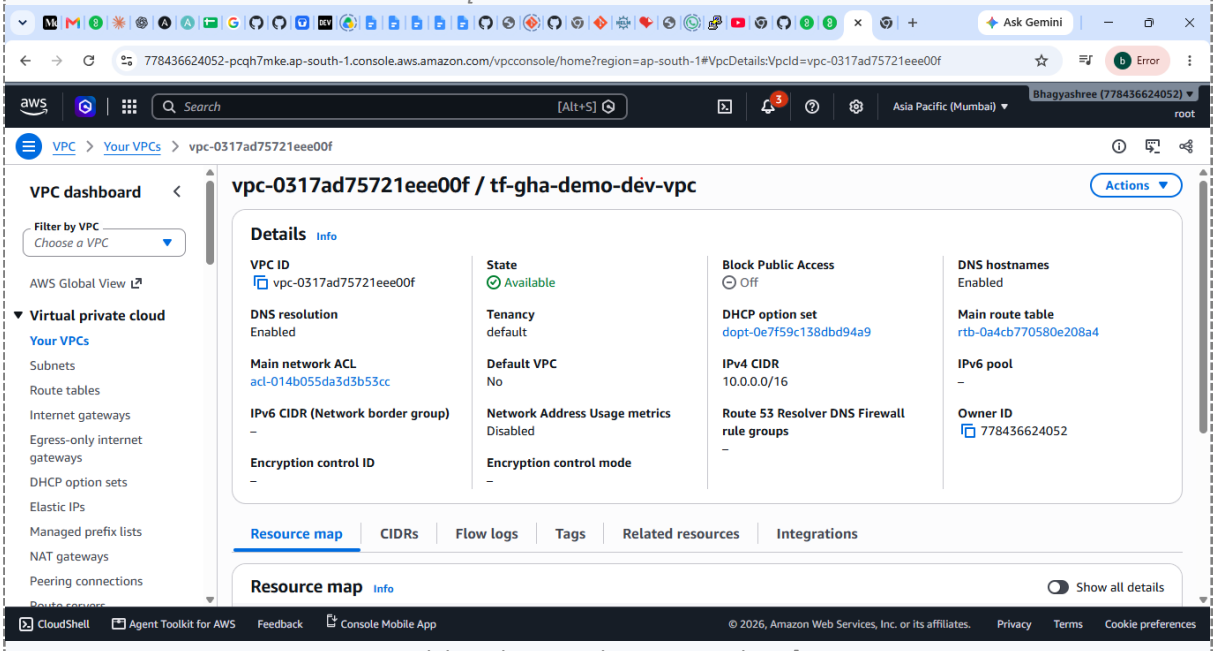
The following screenshots from the AWS Management Console (region ap-south-1, Mumbai) prove that all required resources were provisioned successfully by Terraform.

Screenshot 1: VPC

Where: VPC → Your VPCs

What it shows: The VPC tf-gha-demo-dev-vpc with CIDR 10.0.0.0/16 in the Available state.

[Paste screenshot here —

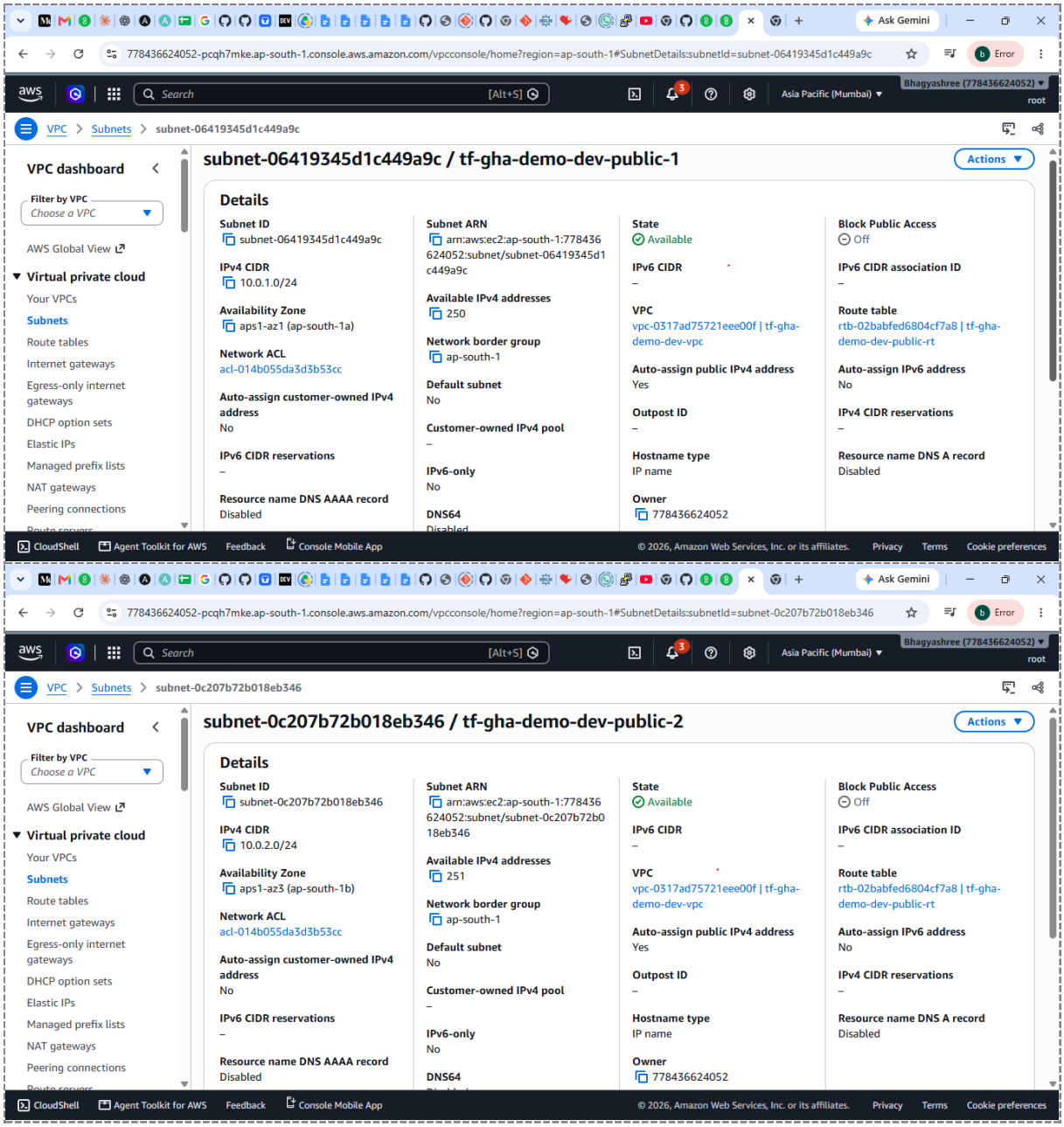


delete this text, then press Ctrl + V]

Screenshot 2: Subnets

Where: VPC → Subnets

What it shows: The two public subnets tf-gha-demo-dev-public-1 (10.0.1.0/24) and tf-gha-demo-dev-public-2 (10.0.2.0/24) in different Availability Zones.

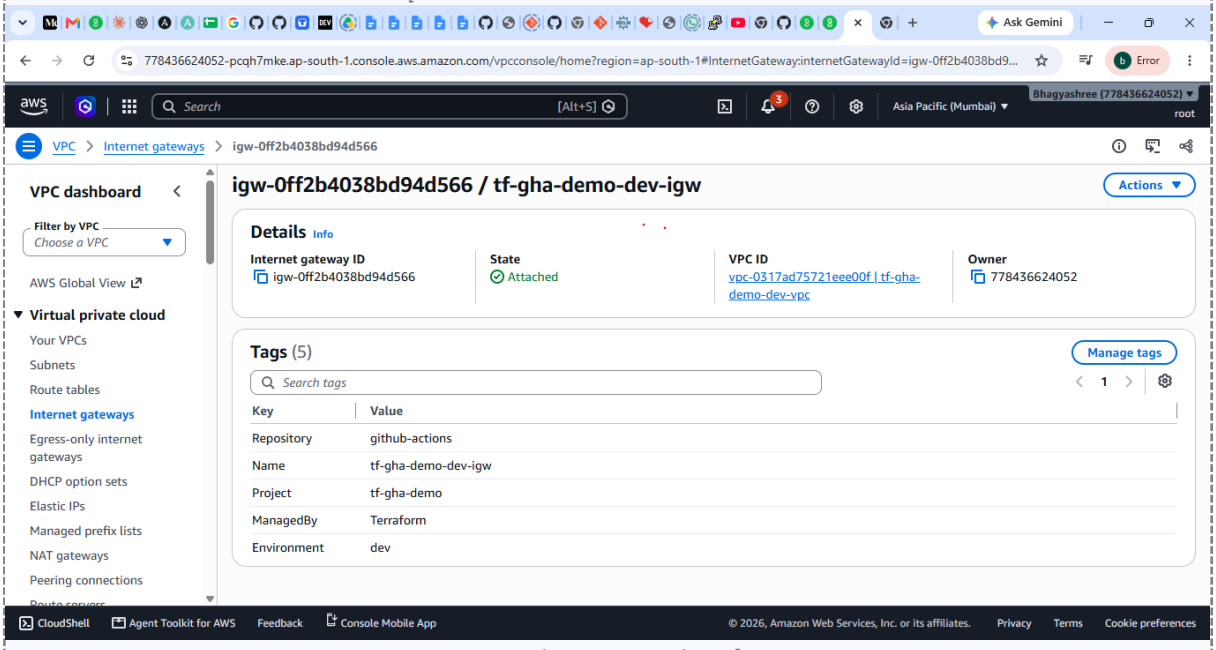


Screenshot 3: Internet Gateway

Where: VPC → Internet gateways

What it shows: The internet gateway tf-gha-demo-dev-igw in the Attached state, connected to the VPC.

[Paste screenshot here — delete this



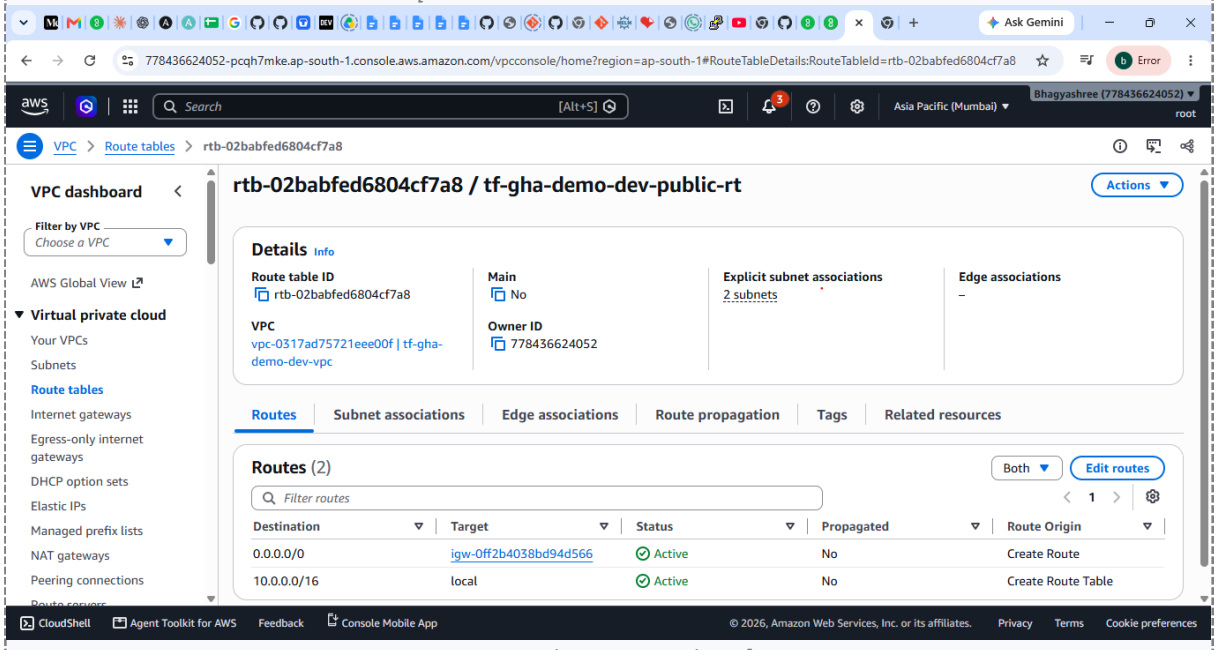
text, then press Ctrl + V]

Screenshot 4: Route Table

Where: VPC → Route tables → tf-gha-demo-dev-public-rt → Routes tab

What it shows: The route 0.0.0.0/0 pointing to the internet gateway, giving the public subnets internet access.

[Paste screenshot here — delete th



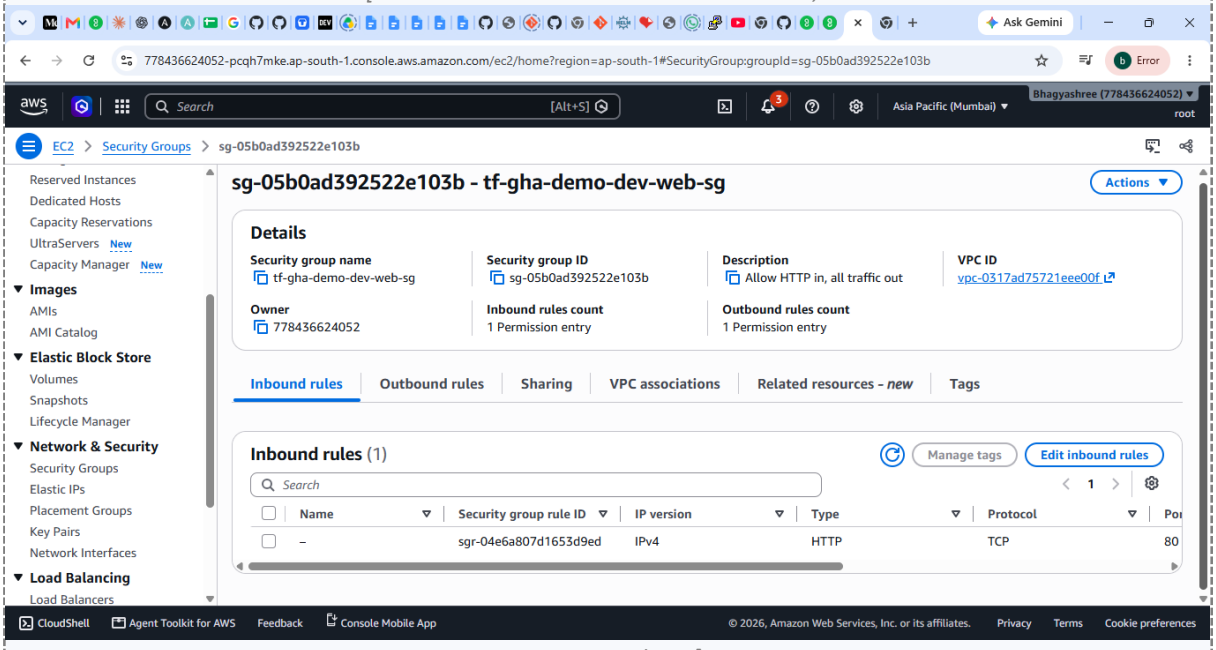
is text, then press Ctrl + V]

Screenshot 5: Security Group

Where: EC2 → Security Groups → tf-gha-demo-dev-web-sg → Inbound rules tab

What it shows: An inbound rule allowing HTTP traffic on port 80.

[Paste screenshot here — delete this text, th



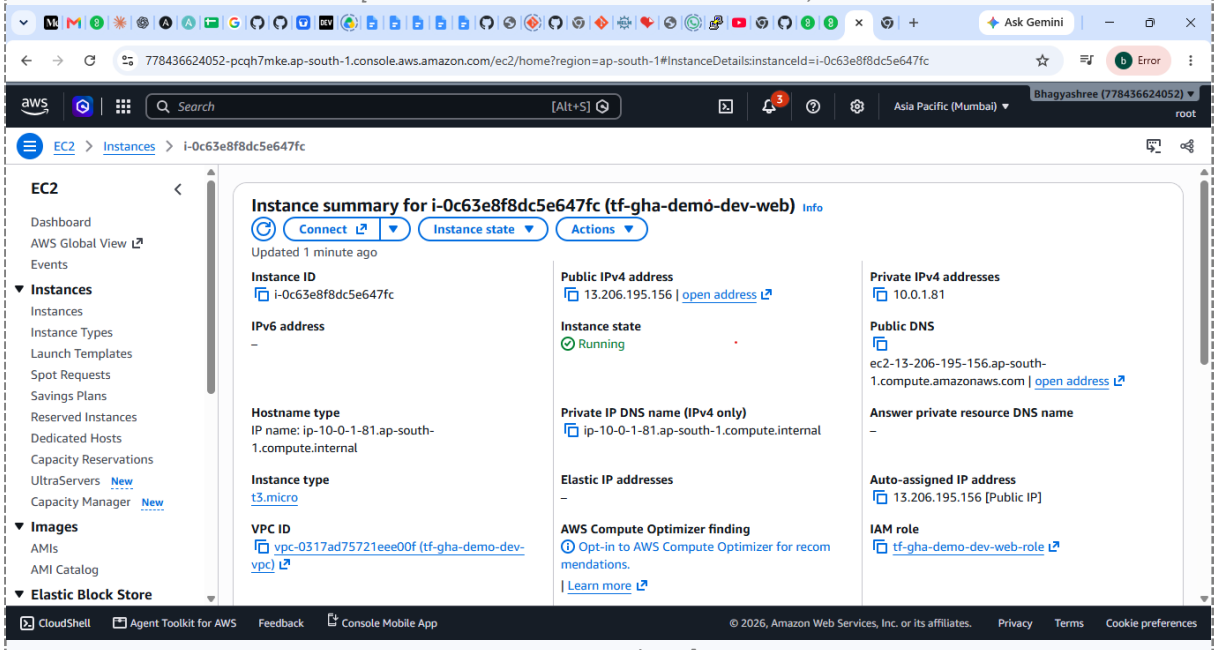
en press Ctrl + V]

Screenshot 6: EC2 Instance

Where: EC2 → Instances

What it shows: The instance tf-gha-demo-dev-web (t3.micro) in the Running state with 2/2 status checks passed.

[Paste screenshot here — delete this text, the



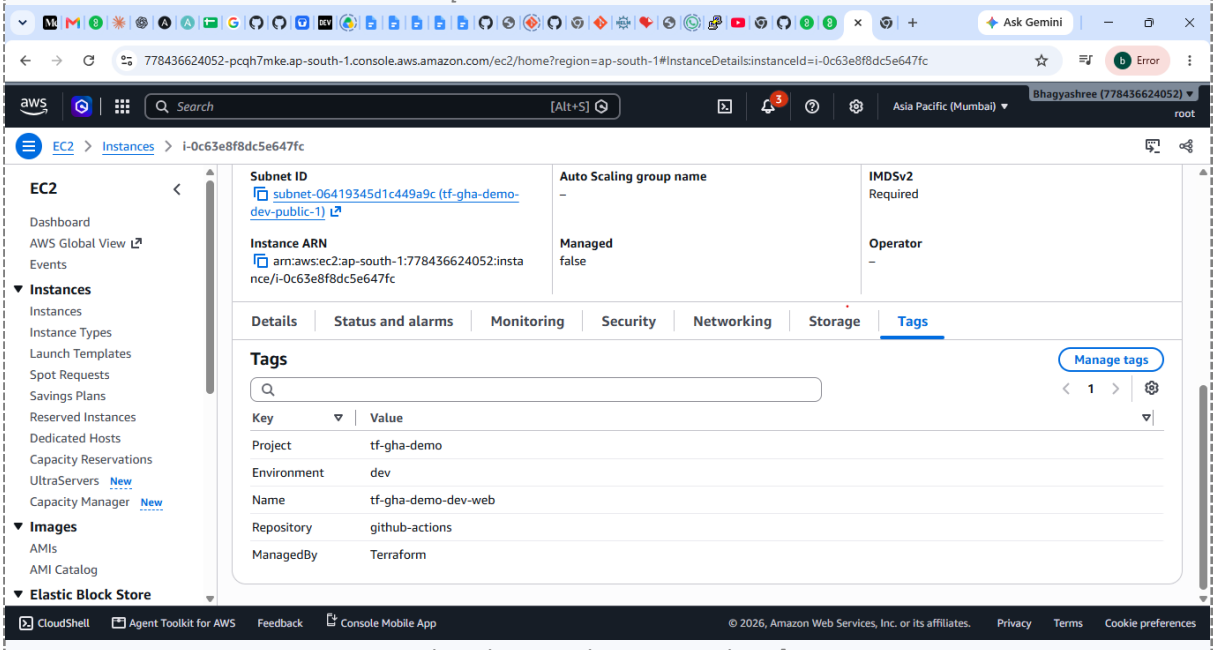
n press Ctrl + V]

Screenshot 7: EC2 Tags (proof of Terraform)

Where: EC2 → Instances → select the instance → Tags tab

What it shows: The tags ManagedBy = Terraform, Project = tf-gha-demo and Environment = dev, proving the resource was created by Terraform.

[Paste screenshot here — de

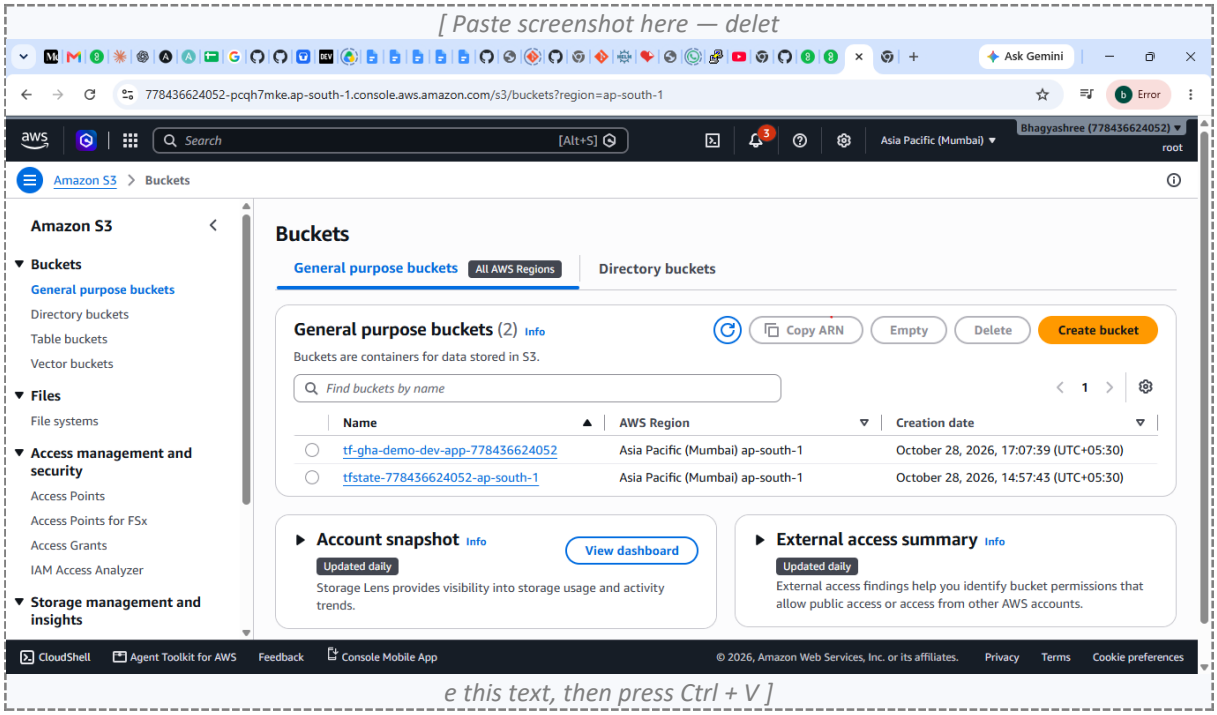


lete this text, then press Ctrl + V]

Screenshot 8: S3 Buckets

Where: S3 → General purpose buckets

What it shows: The application bucket tf-gha-demo-dev-app-… and the Terraform state bucket tfstate-778436624052-ap-south-1.

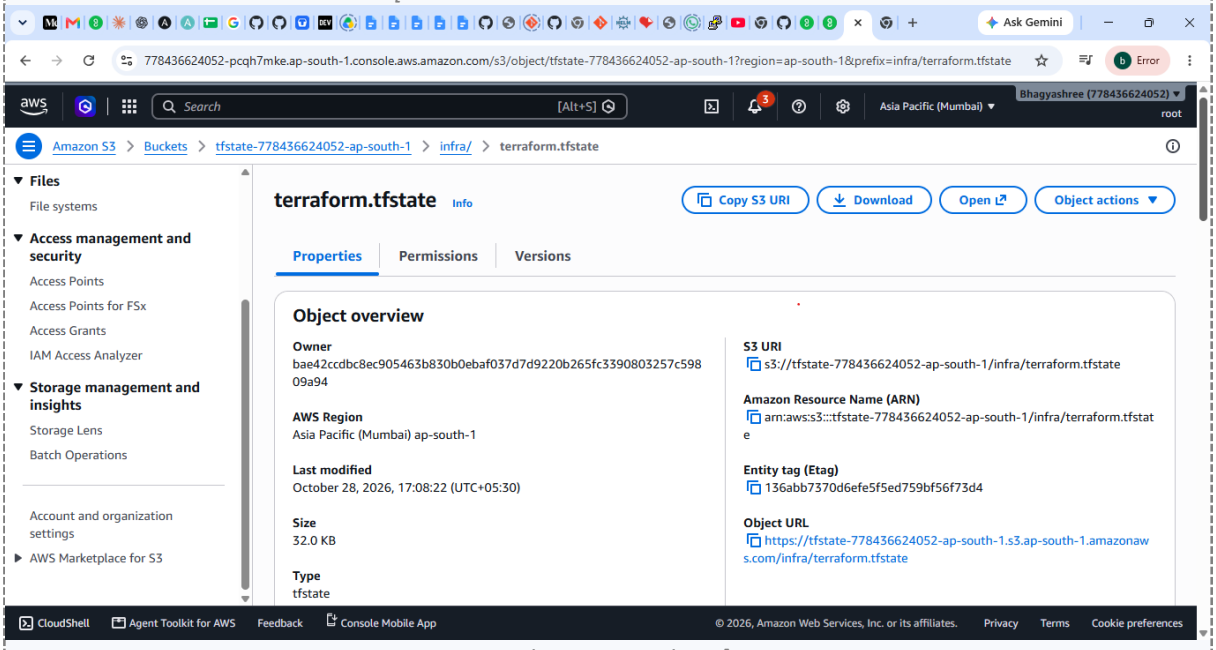


Screenshot 9: Terraform State File

Where: S3 → tfstate-778436624052-ap-south-1 → infra/

What it shows: The terraform.tfstate file stored remotely in S3, where Terraform keeps track of all resources.

[Paste screenshot here — delete this te



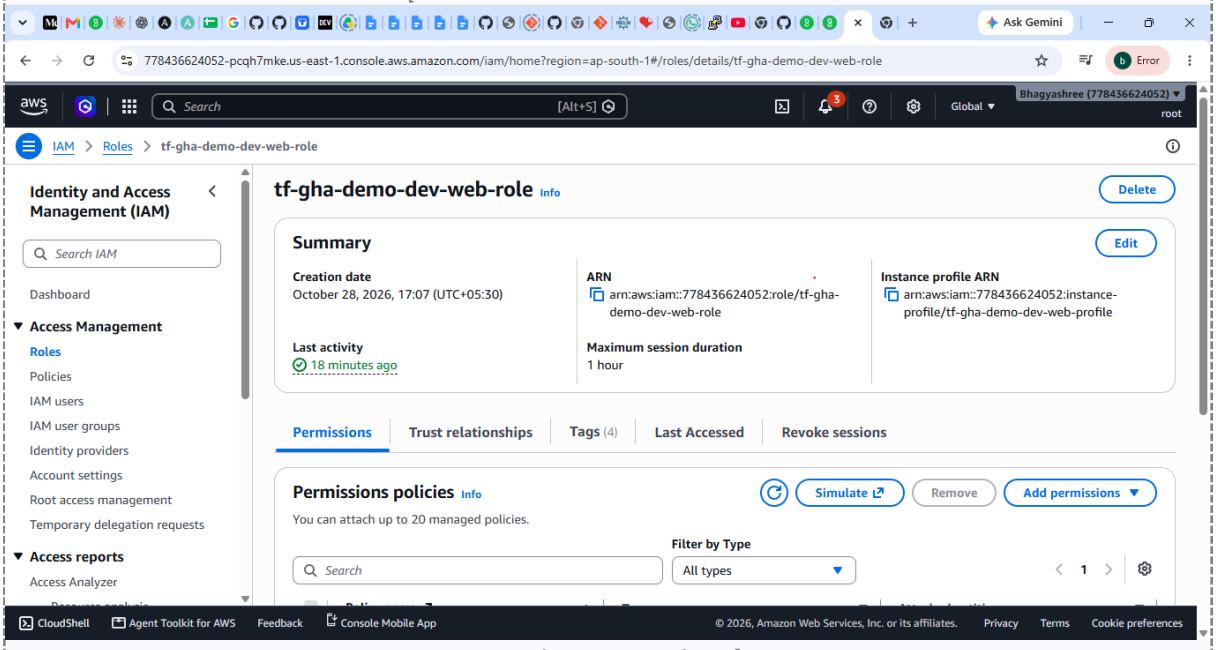
xt, then press Ctrl + V]

Screenshot 10: IAM Role

Where: IAM → Roles → tf-gha-demo-dev-web-role

What it shows: The IAM role attached to the EC2 instance with the AmazonSSMManagedInstanceCore policy.

[Paste screenshot here — delete this



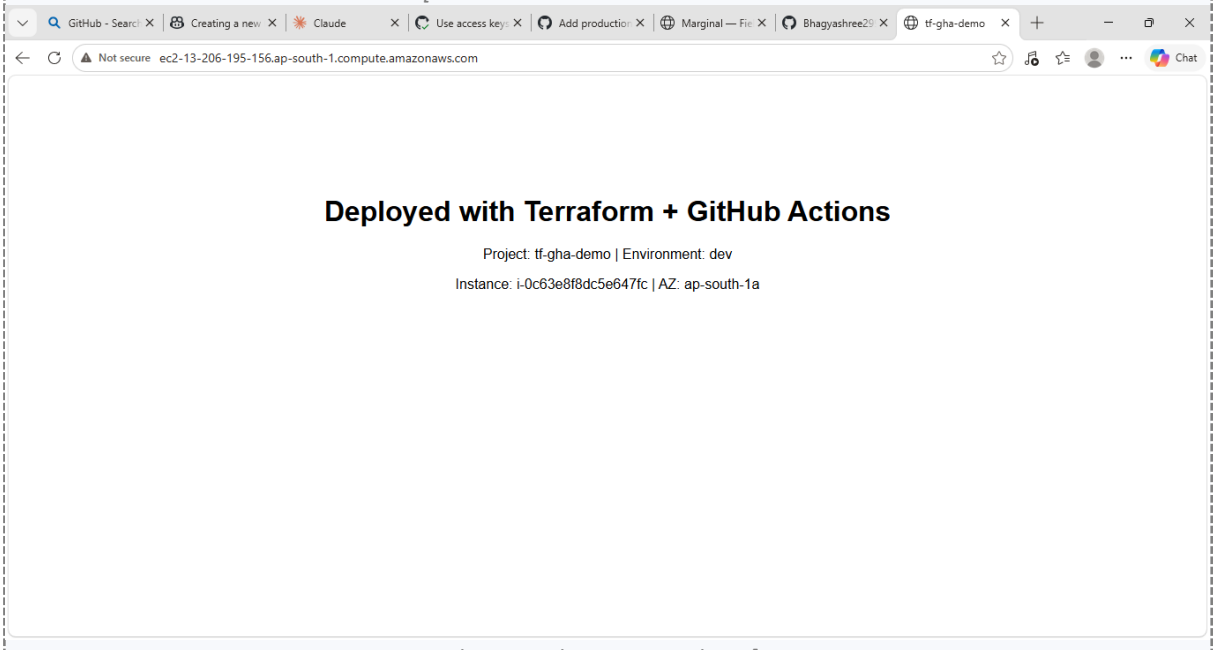
text, then press Ctrl + V]

Screenshot 11: Deployed Website

Where: Open the web_url output in a browser

What it shows: The web page served by nginx on the EC2 instance: "Deployed with Terraform + GitHub Actions".

[Paste screenshot here — delete t



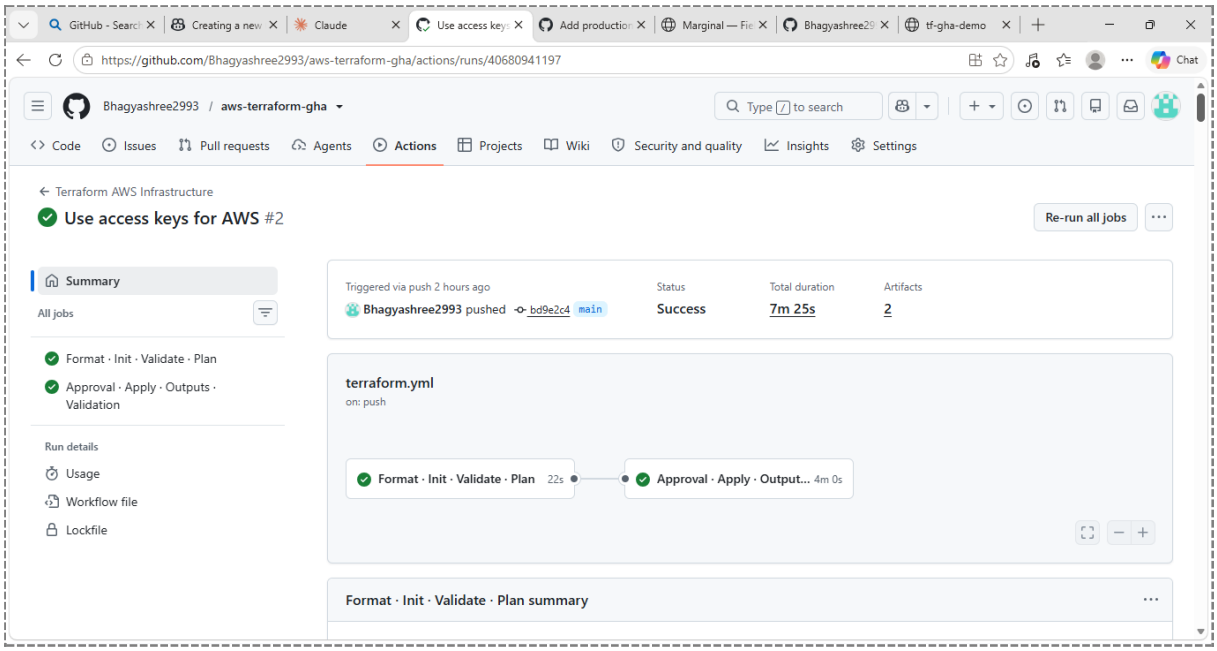
his text, then press Ctrl + V]

6. GitHub Actions Pipeline Evidence

Screenshot 12: Successful Pipeline Run

Where: GitHub → Actions → successful run

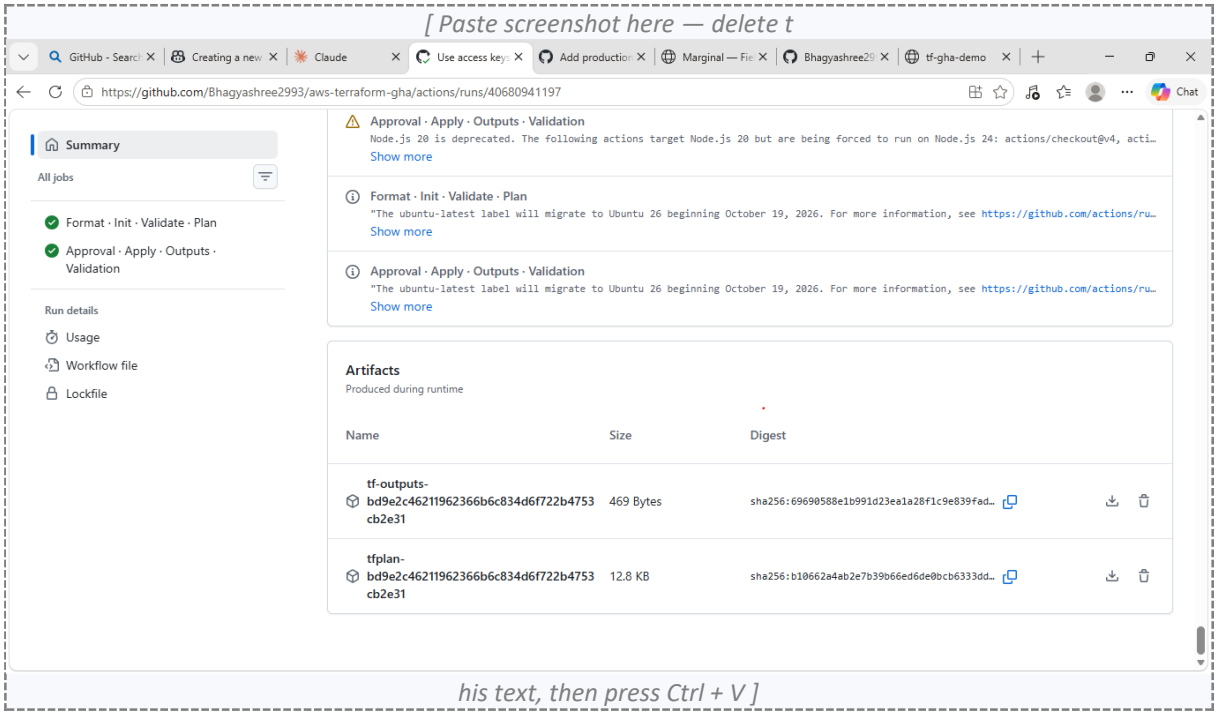
What it shows: Both jobs (Format · Init · Validate · Plan and Approval · Apply · Outputs · Validation) completed with green ticks.



Screenshot 13: Plan Step and Plan Artifact

Where: Actions → run → Summary (scroll down to Artifacts)

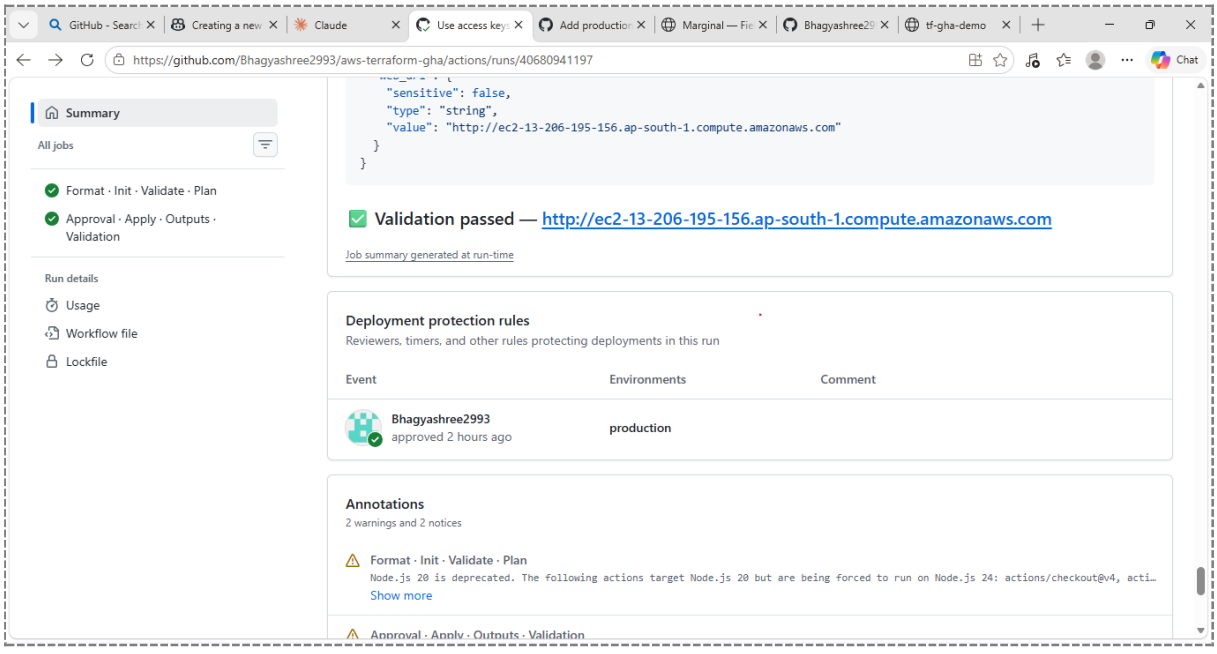
What it shows: The Terraform plan in the run summary and the tfplan artifact uploaded by the pipeline.



Screenshot 14: Manual Approval

Where: Actions → run → Review deployments

What it shows: The pipeline paused at the production environment, waiting for a reviewer to approve before Apply.

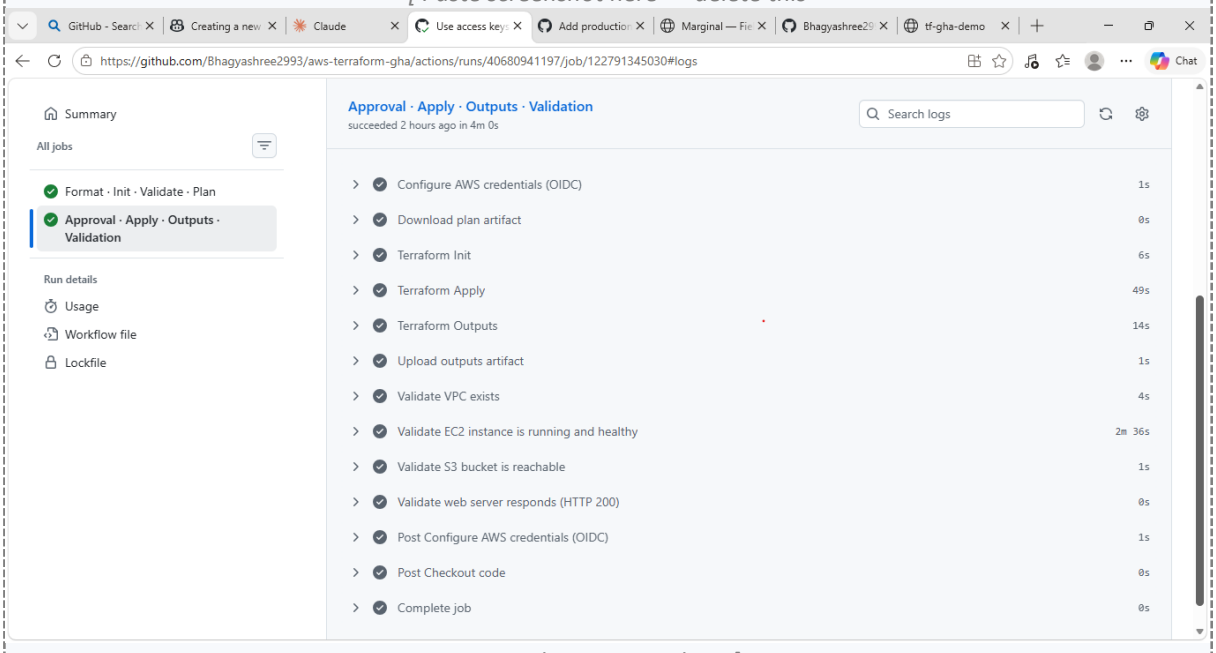


Screenshot 15: Apply, Outputs and Validation

Where: Actions → run → Approval · Apply · Outputs · Validation job / Summary

What it shows: Terraform Apply completed, the Terraform Outputs, and the "Validation passed" message.

[Paste screenshot here — delete this



text, then press Ctrl + V]

7. Terraform Outputs

Values copied from the "Terraform Outputs" section of the run summary:

Output	Value
vpc_id	vpc-0317ad75721eee00f
public_subnet_ids	“subnet-06419345d1c449a9c", "subnet-0c207b72b018eb346"
security_group_id	sg-05b0ad392522e103b
instance_id	i-0c63e8f8dc5e647fc
instance_public_ip	13.206.195.156
web_url	http://ec2-13-206-195-156.ap-south-1.compute.amazonaws.com
app_bucket_name	tf-gha-demo-dev-app-778436624052

8. Security and Best Practices

- Remote state is stored in an encrypted, versioned S3 bucket with state locking, so it is never kept on a laptop and two runs cannot change it at the same time.

- AWS credentials are stored as encrypted GitHub Secrets and are never written in the code.
- A manual approval (GitHub Environment with required reviewers) is required before any change reaches AWS.
- The saved plan file is applied, so exactly what was reviewed is what gets deployed.
- EC2 uses IMDSv2 only and an encrypted root volume, and has no SSH port open (Session Manager is used instead).
- S3 buckets block all public access and use server-side encryption.
- All resources carry tags (Project, Environment, ManagedBy), and a separate Destroy workflow removes everything to avoid charges.

9. Conclusion

All required AWS resources (VPC, public subnets, internet gateway, route table, security group, EC2 web server, IAM role and S3 buckets) were provisioned successfully through Terraform. The deployment was fully automated with a GitHub Actions pipeline that formats, initialises, validates and plans the code, stores the plan as an artifact, waits for manual approval, applies the changes, publishes the outputs and validates the deployed infrastructure. The screenshots in Sections 5 and 6 provide evidence from the AWS Console and GitHub Actions that the infrastructure was created and is working as intended.

Appendix: Terraform Configuration (Source Code)

infra / versions.tf

```
terraform {
  required_version = ">= 1.10.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  # Values (bucket, key, region) are injected at init time by the pipeline:
  #   terraform init -backend-config="bucket=..." -backend-config="key=..." ...
  # use_lockfile enables native S3 state locking (no DynamoDB table needed).
  backend "s3" {
    use_lockfile = true
    encrypt      = true
  }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project     = var.project_name
      Environment = var.environment
      ManagedBy   = "Terraform"
      Repository  = "github-actions"
    }
  }
}
```

infra / variables.tf

```
variable "aws_region" {
  description = "AWS region to deploy into"
  type        = string
  default     = "ap-south-1"
}

variable "project_name" {
  description = "Prefix used in resource names"
  type        = string
  default     = "tf-gha-demo"
}

variable "environment" {
  description = "Deployment environment name"
  type        = string
  default     = "dev"
}

variable "vpc_cidr" {
  description = "CIDR block for the VPC"
  type        = string
  default     = "10.0.0.0/16"

  validation {
    condition     = can(cidrhost(var.vpc_cidr, 0))
    error_message = "vpc_cidr must be a valid IPv4 CIDR block."
  }
}

variable "public_subnet_cidrs" {
  description = "CIDR blocks for public subnets (one per AZ)"
  type        = list(string)
```

```
  default     = ["10.0.1.0/24", "10.0.2.0/24"]
}

variable "instance_type" {
  description = "EC2 instance type for the web server"
  type        = string
  default     = "t3.micro"
}

variable "allowed_http_cidrs" {
  description = "CIDR ranges allowed to reach the web server on port 80"
  type        = list(string)
  default     = ["0.0.0.0/0"]
}
```

infra / terraform.tfvars

```
aws_region          = "ap-south-1"
project_name        = "tf-gha-demo"
environment         = "dev"
vpc_cidr            = "10.0.0.0/16"
public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
instance_type       = "t3.micro"
```

infra / network.tf

```
data "aws_availability_zones" "available" {
  state = "available"
}

locals {
  name = "${var.project_name}-${var.environment}"
}

# ---------------- VPC ----------------
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = { Name = "${local.name}-vpc" }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${local.name}-igw" }
}

# ---------------- Public subnets ----------------
resource "aws_subnet" "public" {
  count = length(var.public_subnet_cidrs)

  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_cidrs[count.index]
  availability_zone       = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true

  tags = { Name = "${local.name}-public-${count.index + 1}" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = { Name = "${local.name}-public-rt" }
}
```

```
resource "aws_route_table_association" "public" {
  count          = length(aws_subnet.public)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

# ---------------- Security group ----------------
resource "aws_security_group" "web" {
  name        = "${local.name}-web-sg"
  description = "Allow HTTP in, all traffic out"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = var.allowed_http_cidrs
  }

  egress {
    description = "All outbound"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "${local.name}-web-sg" }
}
```

infra / compute.tf

```
# Latest Amazon Linux 2023 AMI, resolved through SSM (always current, no hardcoded IDs)
data "aws_ssm_parameter" "al2023" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

# ---------------- IAM role so the instance can use SSM Session Manager (no SSH keys) ----------------
data "aws_iam_policy_document" "ec2_assume" {
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type        = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "web" {
  name               = "${local.name}-web-role"
  assume_role_policy = data.aws_iam_policy_document.ec2_assume.json
}

resource "aws_iam_role_policy_attachment" "ssm" {
  role       = aws_iam_role.web.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}

resource "aws_iam_instance_profile" "web" {
  name = "${local.name}-web-profile"
  role = aws_iam_role.web.name
}

# ---------------- EC2 web server ----------------
resource "aws_instance" "web" {
  ami                    = data.aws_ssm_parameter.al2023.insecure_value
  instance_type          = var.instance_type
  subnet_id              = aws_subnet.public[0].id
```

```
  vpc_security_group_ids = [aws_security_group.web.id]
  iam_instance_profile   = aws_iam_instance_profile.web.name

  metadata_options {
    http_tokens = "required" # IMDSv2 only
  }

  root_block_device {
    volume_size = 8
    volume_type = "gp3"
    encrypted   = true
  }

  user_data = templatefile("${path.module}/user_data.sh.tftpl", {
    project     = var.project_name
    environment = var.environment
  })

  user_data_replace_on_change = true

  tags = { Name = "${local.name}-web" }
}
```

infra / user_data.sh.tftpl

```
#!/bin/bash
set -euxo pipefail

dnf install -y nginx
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-
seconds: 300")
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-
data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-
data/placement/availability-zone)

cat > /usr/share/nginx/html/index.html <<EOF
<!doctype html>
<html>
<head><title>${project}</title></head>
<body style="font-family:sans-serif;text-align:center;margin-top:10%">
  <h1>Deployed with Terraform + GitHub Actions</h1>
  <p>Project: ${project} | Environment: ${environment}</p>
  <p>Instance: $INSTANCE_ID | AZ: $AZ</p>
</body>
</html>
EOF

echo "OK" > /usr/share/nginx/html/health

systemctl enable --now nginx
```

infra / storage.tf

```
data "aws_caller_identity" "current" {}

# Application bucket (private, versioned, encrypted)
resource "aws_s3_bucket" "app" {
  bucket        = "${local.name}-app-${data.aws_caller_identity.current.account_id}"
  force_destroy = true # demo convenience: allows terraform destroy with objects inside

  tags = { Name = "${local.name}-app-bucket" }
}

resource "aws_s3_bucket_versioning" "app" {
  bucket = aws_s3_bucket.app.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

```
resource "aws_s3_bucket_server_side_encryption_configuration" "app" {
  bucket = aws_s3_bucket.app.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "app" {
  bucket                  = aws_s3_bucket.app.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

infra / outputs.tf

```
output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.main.id
}

output "public_subnet_ids" {
  description = "IDs of the public subnets"
  value       = aws_subnet.public[*].id
}

output "security_group_id" {
  description = "ID of the web security group"
  value       = aws_security_group.web.id
}

output "instance_id" {
  description = "ID of the EC2 web server"
  value       = aws_instance.web.id
}

output "instance_public_ip" {
  description = "Public IP of the EC2 web server"
  value       = aws_instance.web.public_ip
}

output "web_url" {
  description = "URL of the deployed web page"
  value       = "http://${aws_instance.web.public_dns}"
}

output "app_bucket_name" {
  description = "Name of the application S3 bucket"
  value       = aws_s3_bucket.app.bucket
}
```

bootstrap / main.tf

```
# ------------------------------------------------------------------------------
# ONE-TIME BOOTSTRAP (run locally with admin credentials, NOT from the pipeline)
# Creates:
#   1. S3 bucket for Terraform remote state (with native S3 state locking)
#   2. GitHub OIDC identity provider in IAM
#   3. IAM role that GitHub Actions assumes (no long-lived AWS keys in GitHub)
# ------------------------------------------------------------------------------

terraform {
  required_version = ">= 1.10.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
```

```
    }
  }
}

provider "aws" {
  region = var.aws_region
}

variable "aws_region" {
  type    = string
  default = "ap-south-1"
}

variable "github_org" {
  description = "GitHub user or organization that owns the repo"
  type        = string
}

variable "github_repo" {
  description = "Repository name (without the owner)"
  type        = string
}

data "aws_caller_identity" "current" {}

# ---------------- Remote state bucket ----------------
resource "aws_s3_bucket" "tf_state" {
  bucket = "tfstate-${data.aws_caller_identity.current.account_id}-${var.aws_region}"

  lifecycle {
    prevent_destroy = true
  }
}

resource "aws_s3_bucket_versioning" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "tf_state" {
  bucket                  = aws_s3_bucket.tf_state.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# ---------------- GitHub OIDC ----------------
resource "aws_iam_openid_connect_provider" "github" {
  url             = "https://token.actions.githubusercontent.com"
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

data "aws_iam_policy_document" "github_trust" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
```

```
    principals {
      type        = "Federated"
      identifiers = [aws_iam_openid_connect_provider.github.arn]
    }

    condition {
      test     = "StringEquals"
      variable = "token.actions.githubusercontent.com:aud"
      values   = ["sts.amazonaws.com"]
    }

    # Only this repository may assume the role
    condition {
      test     = "StringLike"
      variable = "token.actions.githubusercontent.com:sub"
      values   = ["repo:${var.github_org}/${var.github_repo}:*"]
    }
  }
}

resource "aws_iam_role" "github_actions" {
  name               = "github-actions-terraform"
  assume_role_policy = data.aws_iam_policy_document.github_trust.json
}

# For a lab/demo, PowerUser + limited IAM is enough. Tighten for production.
resource "aws_iam_role_policy_attachment" "poweruser" {
  role       = aws_iam_role.github_actions.name
  policy_arn = "arn:aws:iam::aws:policy/PowerUserAccess"
}

output "state_bucket_name" {
  value = aws_s3_bucket.tf_state.bucket
}

output "github_actions_role_arn" {
  value = aws_iam_role.github_actions.arn
}
```

.github / workflows / terraform.yml

```
# GitHub → GitHub Actions → Format → Init → Validate → Plan → Plan Artifact
#        → Approval → Apply → AWS Infrastructure → Outputs → Validation
name: Terraform AWS Infrastructure

on:
  push:
    branches: [main]
    paths: ["infra/**", ".github/workflows/terraform.yml"]
  pull_request:
    branches: [main]
    paths: ["infra/**", ".github/workflows/terraform.yml"]
  workflow_dispatch:

permissions:
  id-token: write       # required for AWS OIDC
  contents: read
  pull-requests: write  # to comment the plan on PRs

concurrency:
  group: terraform-${{ github.ref }}
  cancel-in-progress: false

env:
  TF_VERSION: "1.13.4"
  AWS_REGION: ${{ vars.AWS_REGION }}
  TF_WORKING_DIR: infra
  TF_IN_AUTOMATION: "true"
```

```
jobs:
  # ======================================================================
  # Stages 1-5: Format → Init → Validate → Plan → Plan Artifact
  # ======================================================================
  plan:
    name: Format · Init · Validate · Plan
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ${{ env.TF_WORKING_DIR }}
    outputs:
      has_changes: ${{ steps.plan.outputs.has_changes }}

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}
          terraform_wrapper: false

      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      # ---------- 1. Format ----------
      - name: Terraform Format
        id: fmt
        run: terraform fmt -check -recursive -diff

      # ---------- 2. Init ----------
      - name: Terraform Init
        id: init
        run: |
          terraform init -input=false \
            -backend-config="bucket=${{ secrets.TF_STATE_BUCKET }}" \
            -backend-config="key=infra/terraform.tfstate" \
            -backend-config="region=${{ env.AWS_REGION }}"

      # ---------- 3. Validate ----------
      - name: Terraform Validate
        id: validate
        run: terraform validate -no-color

      # ---------- 4. Plan ----------
      - name: Terraform Plan
        id: plan
        run: |
          set +e
          terraform plan -input=false -no-color -detailed-exitcode -out=tfplan
          code=$?
          set -e
          # exit code: 0 = no changes, 1 = error, 2 = changes present
          if [ $code -eq 1 ]; then exit 1; fi
          if [ $code -eq 2 ]; then echo "has_changes=true" >> "$GITHUB_OUTPUT"; else echo
"has_changes=false" >> "$GITHUB_OUTPUT"; fi
          terraform show -no-color tfplan > plan.txt

      - name: Plan summary
        run: |
          {
            echo "## Terraform Plan"
            echo "Changes detected: **${{ steps.plan.outputs.has_changes }}**"
            echo '```hcl'
```

```
            head -c 60000 plan.txt
            echo '```'
          } >> "$GITHUB_STEP_SUMMARY"

      - name: Comment plan on PR
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const plan = fs.readFileSync('${{ env.TF_WORKING_DIR }}/plan.txt', 'utf8').slice(0,
60000);
            const body = `### Terraform Plan 📋
            Format: \`${{ steps.fmt.outcome }}\` · Init: \`${{ steps.init.outcome }}\` · Validate:
\`${{ steps.validate.outcome }}\` · Plan: \`${{ steps.plan.outcome }}\`

            <details><summary>Show plan</summary>

            \`\`\`hcl
            ${plan}
            \`\`\`
            </details>`;
            await github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body
            });

      # ---------- 5. Plan Artifact ----------
      - name: Upload plan artifact
        uses: actions/upload-artifact@v4
        with:
          name: tfplan-${{ github.sha }}
          path: |
            ${{ env.TF_WORKING_DIR }}/tfplan
            ${{ env.TF_WORKING_DIR }}/plan.txt
            ${{ env.TF_WORKING_DIR }}/.terraform.lock.hcl
          retention-days: 5

  # =====================================================================
  # Stages 6-10: Approval → Apply → AWS Infrastructure → Outputs → Validation
  # Runs only on main (push / manual), and only when the plan has changes.
  # =====================================================================
  apply:
    name: Approval · Apply · Outputs · Validation
    needs: plan
    if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request' &&
needs.plan.outputs.has_changes == 'true'
    runs-on: ubuntu-latest

    # ---------- 6. Approval ----------
    # The "production" environment has Required Reviewers configured in
    # Settings → Environments. The job pauses here until someone approves.
    environment:
      name: production

    defaults:
      run:
        working-directory: ${{ env.TF_WORKING_DIR }}

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}
          terraform_wrapper: false
```

```
      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Download plan artifact
        uses: actions/download-artifact@v4
        with:
          name: tfplan-${{ github.sha }}
          path: ${{ env.TF_WORKING_DIR }}

      - name: Terraform Init
        run: |
          terraform init -input=false \
            -backend-config="bucket=${{ secrets.TF_STATE_BUCKET }}" \
            -backend-config="key=infra/terraform.tfstate" \
            -backend-config="region=${{ env.AWS_REGION }}"

      # ---------- 7. Apply → 8. AWS Infrastructure ----------
      # Applies exactly the reviewed plan; fails if state changed since planning.
      - name: Terraform Apply
        run: terraform apply -input=false -auto-approve tfplan

      # ---------- 9. Outputs ----------
      - name: Terraform Outputs
        id: outputs
        run: |
          terraform output -json > outputs.json
          echo "web_url=$(terraform output -raw web_url)"           >> "$GITHUB_OUTPUT"
          echo "instance_id=$(terraform output -raw instance_id)"   >> "$GITHUB_OUTPUT"
          echo "bucket=$(terraform output -raw app_bucket_name)"    >> "$GITHUB_OUTPUT"
          echo "vpc_id=$(terraform output -raw vpc_id)"             >> "$GITHUB_OUTPUT"
          {
            echo "## Terraform Outputs"
            echo '```json'
            cat outputs.json
            echo '```'
          } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload outputs artifact
        uses: actions/upload-artifact@v4
        with:
          name: tf-outputs-${{ github.sha }}
          path: ${{ env.TF_WORKING_DIR }}/outputs.json

      # ---------- 10. Validation ----------
      - name: Validate VPC exists
        run: |
          state=$(aws ec2 describe-vpcs --vpc-ids "${{ steps.outputs.outputs.vpc_id }}" \
                   --query 'Vpcs[0].State' --output text)
          echo "VPC state: $state"
          [ "$state" = "available" ]

      - name: Validate EC2 instance is running and healthy
        run: |
          aws ec2 wait instance-status-ok --instance-ids "${{ steps.outputs.outputs.instance_id }}"
          aws ec2 describe-instances --instance-ids "${{ steps.outputs.outputs.instance_id }}" \
            --query 'Reservations[0].Instances[0].[InstanceId,State.Name,PublicIpAddress]' --output
table

      - name: Validate S3 bucket is reachable
        run: aws s3api head-bucket --bucket "${{ steps.outputs.outputs.bucket }}"

      - name: Validate web server responds (HTTP 200)
        run: |
          url="${{ steps.outputs.outputs.web_url }}/health"
          for i in $(seq 1 30); do
```

```
            code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "$url" || true)
            echo "Attempt $i: HTTP $code"
            if [ "$code" = "200" ]; then
              echo "✅ Web server is up at ${{ steps.outputs.outputs.web_url }}"
              echo "### ✅ Validation passed — ${{ steps.outputs.outputs.web_url }}" >>
"$GITHUB_STEP_SUMMARY"
              exit 0
            fi
            sleep 10
          done
          echo "❌ Web server did not return 200 within 5 minutes"
          exit 1
```

.github / workflows / terraform-destroy.yml

```
name: Terraform Destroy

on:
  workflow_dispatch:
    inputs:
      confirm:
        description: 'Type "destroy" to confirm tearing down all infrastructure'
        required: true

permissions:
  id-token: write
  contents: read

env:
  TF_VERSION: "1.13.4"
  AWS_REGION: ${{ vars.AWS_REGION }}

jobs:
  destroy:
    if: github.event.inputs.confirm == 'destroy'
    runs-on: ubuntu-latest
    environment: production   # same approval gate as apply
    defaults:
      run:
        working-directory: infra
    steps:
      - uses: actions/checkout@v4

      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}

      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Terraform Init
        run: |
          terraform init -input=false \
            -backend-config="bucket=${{ secrets.TF_STATE_BUCKET }}" \
            -backend-config="key=infra/terraform.tfstate" \
            -backend-config="region=${{ env.AWS_REGION }}"

      - name: Terraform Destroy
        run: terraform destroy -input=false -auto-approve
```